

Les nombres en notation scientifique

Un ordinateur ne peut traiter les nombres de la même manière que l'être humain. Il doit d'abord les **représenter** dans un système qui permet l'exécution **efficace** des diverses opérations. Cela peut entraîner des **erreurs** de représentation sur ordinateur, qui sont inévitables et dont il est souhaitable de comprendre l'origine afin de mieux en maîtriser les effets.

1. Préliminaires

Dans cette partie on vérifie comment **Python** fonctionne, puis on introduira les boucles (**for** et **while**), le test **if ... else ...** et les fonctions.

1.1. Premiers pas avec Python

Pour commencer testons si tout fonctionne !

Travaux pratiques 1.

1. Définir deux variables prenant les valeurs 3 et 6.
2. Calculer leur somme et leur produit.

Voici à quoi cela ressemble :

Code 1 (*hello-world.py*).

```
>>> a=3
>>> b=6
>>> somme = a+b
>>> print(somme)
9
>>> # Les résultats
>>> print("La somme est", somme)
La somme est 9
>>> produit = a*b
>>> print("Le produit est", produit)
Le produit est 18
```

On retient les choses suivantes :

- On affecte une valeur à une variable par le signe égal =.
- On affiche un message avec la fonction **print()**.

- Lorsque qu'une ligne contient un dièse #, tout ce qui suit est ignoré. Cela permet d'insérer des commentaires, ce qui est essentiel pour relire le code.

Dans la suite on omettra les symboles >>>. Voir plus de détails sur le fonctionnement en fin de section.

1.2. Somme des cubes

Travaux pratiques 2.

1. Pour un entier n fixé, programmer le calcul de la somme $S_n = 1^3 + 2^3 + 3^3 + \dots + n^3$.
2. Définir une fonction qui pour une valeur n renvoie la somme $\Sigma_n = 1 + 2 + 3 + \dots + n$.
3. Définir une fonction qui pour une valeur n renvoie S_n .
4. Vérifier, pour les premiers entiers, que $S_n = (\Sigma_n)^2$.

1.

Code 2 (*somme-cubes.py* (1)).

```
n = 10
somme = 0
for i in range(1,n+1):
    somme = somme + i*i*i
print(somme)
```

Voici ce que l'on fait pour calculer S_n avec $n = 10$.

- On affecte d'abord la valeur 0 à la variable `somme`, cela correspond à l'initialisation $S_0 = 0$.
 - Nous avons défini une **boucle** avec l'instruction **for** qui fait varier i entre 1 et n .
 - Nous calculons successivement $S_1, S_2, \dots$ en utilisant la formule de récurrence $S_i = S_{i-1} + i^3$. Comme nous n'avons pas besoin de conserver toutes les valeurs des S_i alors on garde le même nom pour toutes les sommes, à chaque étape on affecte à `somme` l'ancienne valeur de la somme plus i^3 : `somme = somme + i*i*i`.
 - `range(1,n+1)` est l'ensemble des entiers $\{1, 2, \dots, n\}$. C'est bien les entiers **strictement inférieurs à** $n + 1$. La raison est que `range(n)` désigne $\{0, 1, 2, \dots, n - 1\}$ qui contient n éléments.
2. Nous savons que $\Sigma_n = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$ donc nous n'avons pas besoin de faire une boucle :

Code 3 (*somme-cubes.py* (2)).

```
def somme_entiers(n):
    return n*(n+1)/2
```

Une **fonction** en informatique est similaire à une fonction mathématique, c'est un objet qui prend en entrée des variables (dites variables formelles ou variables muettes, ici n) et retourne une valeur (un entier, une liste, une chaîne de caractères,... ici $\frac{n(n+1)}{2}$).

3. Voici la fonction qui retourne la somme des cubes :

Code 4 (*somme-cubes.py* (3)).

```
def somme_cubes(n):
    somme = 0
    for i in range(1,n+1):
        somme = somme + i**3
    return somme
```

4. Et enfin on vérifie que pour les premiers entiers $S_n = \left(\frac{n(n+1)}{2}\right)^2$, par exemple pour $n = 12$:

Code 5 (*somme-cubes.py* (4)).

```
n = 12
if somme_cubes(n) == (somme_entiers(n)**2):
    print("Pour n=", n, "l'assertion est vraie.")
else:
    print("L'assertion est fausse!")
```

On retient :

- Les puissances se calculent aussi avec ****** : 5^2 s'écrit `5*5` ou `5**2`, 5^3 s'écrit `5*5*5` ou `5**3`,...
- Une fonction se définit par **def** `ma_fonction(variable):` et se termine par **return** `resultat`.
- **if condition: ... else: ...** exécute le premier bloc d'instructions si la condition est vraie ; si la condition est fausse cela exécute l'autre bloc.
- Exemple de conditions
 - `a < b` : $a < b$,
 - `a <= b` : $a \leq b$,
 - `a == b` : $a = b$,
 - `a != b` : $a \neq b$.
- Attention ! Il est important de comprendre que `a==b` vaut soit vraie ou faux (on compare a et b) alors qu'avec `a=b` on affecte dans a la valeur de b .
- Enfin en **Python** (contrairement aux autres langages) c'est l'indentation (les espaces en début de chaque ligne) qui détermine les blocs d'instructions.

1.3. Un peu plus sur Python

- Le plus surprenant avec **Python** c'est que c'est *l'indentation* qui détermine le début et la fin d'un bloc d'instructions. Cela oblige à présenter très soigneusement le code.
- Contrairement à d'autres langages on n'a pas besoin de déclarer le type de variable. Par exemple lorsque l'on initialise une variable par `x=0`, on n'a pas besoin de préciser si x est un entier ou un réel.
- La première façon de lancer **Python** est en ligne de commande, on obtient alors l'invite `>>>` et on tape les commandes.
- Mais le plus pratique est de sauvegarder ses commandes dans un fichier et de faire un appel par `python monfichier.py`
- Vous trouverez sans problème de l'aide et des tutoriels sur internet !

1.4. Division euclidienne et reste, calcul avec les modulo

La division euclidienne de a par b , avec $a \in \mathbb{Z}$ et $b \in \mathbb{Z}^*$ s'écrit :

$$a = bq + r \quad \text{et} \quad 0 \leq r < b$$

où $q \in \mathbb{Z}$ est le *quotient* et $r \in \mathbb{N}$ est le *reste*.

En **Python** le quotient se calcule par `a // b`. Le reste se calcule par `a % b`. Exemple : `14 // 3` retourne 4 alors que `14 % 3` (lire 14 modulo 3) retourne 2. On a bien $14 = 3 \times 4 + 2$.

Les calculs avec les modulus sont très pratiques. Par exemple si l'on souhaite tester si un entier est pair, ou impair cela revient à un test modulo 2. Le code est `if (n%2 == 0): ... else: ....` Si on besoin de calculer $\cos(n\frac{\pi}{2})$ alors il faut discuter suivant les valeurs de `n%4`.

Appliquons ceci au problème suivant :

Travaux pratiques 3.

Combien y-a-t-il d'occurrences du chiffre 1 dans les nombres de 1 à 999 ? Par exemple le chiffre 1 apparaît une fois dans 51 mais deux fois dans 131.

Code 6 (*nb-un.py*).

```
NbDeUn = 0
for N in range(1,999+1):
    ChiffreUnite = N % 10
    ChiffreDizaine = (N // 10) % 10
    ChiffreCentaine = (N // 100) % 10
    if (ChiffreUnite == 1):
        NbDeUn = NbDeUn + 1
    if (ChiffreDizaine == 1):
        NbDeUn = NbDeUn + 1
    if (ChiffreCentaine == 1):
        NbDeUn = NbDeUn + 1
print("Nombre d'occurrences du chiffre '1' :", NbDeUn)
```

Commentaires :

- Comment obtient-on le chiffre des unités d'un entier N ? C'est le reste modulo 10, d'où l'instruction `ChiffreUnite = N % 10`.
- Comment obtient-on le chiffre des dizaines ? C'est plus délicat, on commence par effectuer la division euclidienne de N par 10 (cela revient à supprimer le chiffre des unités, par exemple si $N = 251$ alors $N // 10$ retourne 25). Il ne reste plus qu'à calculer le reste modulo 10, (par exemple $(N // 10) \% 10$ retourne le chiffre des dizaines 5).
- Pour le chiffre des centaines on divise d'abord par 100.

1.5. Écriture des nombres en base 10

L'écriture décimale d'un nombre, c'est associer à un entier N la suite de ses chiffres $[a_0, a_1, \dots, a_n]$ de sorte que a_i soit le i -ème chiffre de N . C'est-à-dire

$$N = a_n 10^n + a_{n-1} 10^{n-1} + \dots + a_2 10^2 + a_1 10 + a_0 \quad \text{et } a_i \in \{0, 1, \dots, 9\}$$

a_0 est le chiffre des unités, a_1 celui des dizaines, a_2 celui des centaines,...

Travaux pratiques 4.

1. Écrire une fonction qui à partir d'une liste $[a_0, a_1, \dots, a_n]$ calcule l'entier N correspondant.
2. Pour un entier N fixé, combien a-t-il de chiffres ? On pourra s'aider d'une inégalité du type $10^n \leq N < 10^{n+1}$.
3. Écrire une fonction qui à partir de N calcule son écriture décimale $[a_0, a_1, \dots, a_n]$.

Voici le premier algorithme :

Code 7 (*decimale.py (1)*).

```
def chiffres_vers_entier(tab):
    N = 0
    for i in range(len(tab)):
        N = N + tab[i] * (10 ** i)
    return N
```

La formule mathématique est simplement $N = a_n 10^n + a_{n-1} 10^{n-1} + \dots + a_2 10^2 + a_1 10 + a_0$. Par exemple `chiffres_vers_entier([4,3,2,1])` renvoie l'entier 1234.

Expliquons les bases sur les *listes* (qui s'appelle aussi des *tableaux*)

- En Python une liste est présentée entre des crochets. Par exemple pour `tab = [4,3,2,1]` alors on accède aux valeurs par `tab[i]` : `tab[0]` vaut 4, `tab[1]` vaut 3, `tab[2]` vaut 2, `tab[3]` vaut 1.
- Pour parcourir les éléments d'un tableau le code est simplement `for x in tab`, `x` vaut alors successivement 4, 3, 2, 1.
- La longueur du tableau s'obtient par `len(tab)`. Pour notre exemple `len([4,3,2,1])` vaut 4. Pour parcourir toutes les valeurs d'un tableau on peut donc aussi écrire `for i in range(len(tab))`, puis utiliser `tab[i]`, ici `i` variant ici de 0 à 3.
- La liste vide est seulement notée avec deux crochets : `[]`. Elle est utile pour initialiser une liste.
- Pour ajouter un élément à une liste `tab` existante on utilise la fonction `append`. Par exemple définissons la liste vide `tab=[]`, pour ajouter une valeur à la fin de la liste on saisit : `tab.append(4)`. Maintenant notre liste est `[4]`, elle contient un seul élément. Si on continue avec `tab.append(3)`. Alors maintenant notre liste a deux éléments : `[4,3]`.

Voici l'écriture d'un entier en base 10 :

Code 8 (*decimale.py* (2)).

```
def entier_vers_chiffres(N):
    tab = []
    n = floor(log(N,10)) # le nombre de chiffres est n+1
    for i in range(0,n+1):
        tab.append((N // 10 ** i) % 10)
    return tab
```

Par exemple `entier_vers_chiffres(1234)` renvoie le tableau `[4,3,2,1]`. Nous avons expliqué tout ce dont nous avons besoin sur les listes au-dessus, expliquons les mathématiques.

- Décomposons $\mathbb{N}^*$ sous la forme $[1, 10[\cup [10, 100[\cup [100, 1000[\cup [1000, 10000[\cup \dots$. Chaque intervalle est du type $[10^n, 10^{n+1}[$. Pour $N \in \mathbb{N}^*$ il existe donc $n \in \mathbb{N}$ tel que $10^n \leq N < 10^{n+1}$. Ce qui indique que le nombre de chiffres de N est $n + 1$.

Par exemple si $N = 1234$ alors $1000 = 10^3 \leq N < 10^4 = 10000$, ainsi $n = 3$ et le nombre de chiffres est 4.

- Comment calculer n à partir de N ? Nous allons utiliser le logarithme décimal $\log_{10}$ qui vérifie $\log_{10}(10) = 1$ et $\log_{10}(10^i) = i$. Le logarithme est une fonction croissante, donc l'inégalité $10^n \leq N < 10^{n+1}$ devient $\log_{10}(10^n) \leq \log_{10}(N) < \log_{10}(10^{n+1})$. Et donc $n \leq \log_{10}(N) < n + 1$. Ce qui indique donc que $n = E(\log_{10}(N))$ où $E(x)$ désigne la partie entière d'un réel x .

1.6. Module math

Quelques commentaires informatiques sur un module important pour nous. Les fonctions mathématiques ne sont pas définies par défaut dans Python (à part $|x|$ et x^n), il faut faire appel à une librairie spéciale : le module `math` contient les fonctions mathématiques principales.

<code>abs(x)</code>	$ x $
<code>x ** n</code>	x^n
<code>sqrt(x)</code>	$\sqrt{x}$
<code>exp(x)</code>	$\exp x$
<code>log(x)</code>	$\ln x$ logarithme népérien
<code>log(x,10)</code>	$\log x$ logarithme décimal
<code>cos(x)</code> , <code>sin(x)</code> , <code>tan(x)</code>	$\cos x$, $\sin x$, $\tan x$ en radians
<code>acos(x)</code> , <code>asin(x)</code> , <code>atan(x)</code>	$\arccos x$, $\arcsin x$, $\arctan x$ en radians
<code>floor(x)</code>	partie entière $E(x)$: plus grand entier $n \leq x$ (<i>floor</i> = plancher)
<code>ceil(x)</code>	plus petit entier $n \geq x$ (<i>ceil</i> = plafond)

- Comme on aura souvent besoin de ce module on l'appelle par le code `from math import *`. Cela signifie que l'on importe toutes les fonctions de ce module et qu'en plus on n'a pas besoin de préciser que la fonction vient du module `math`. On peut écrire `cos(3.14)` au lieu `math.cos(3.14)`.
- Dans l'algorithme précédent nous avons utilisé le logarithme décimal `log(x,10)`, ainsi que la partie entière `floor(x)`.

2. Représentation des nombres

2.1. Les réels

En mathématique un réel est un élément de $\mathbb{R}$ et son écriture décimale est souvent *infinie* après la virgule : par exemple $\pi = 3,14159265\dots$. Mais bien sûr un ordinateur ne peut pas coder une infinité d'informations. Ce qui se rapproche d'un réel est un *nombre flottant* dont l'écriture est :

$$\underbrace{\pm 1,234567890123456789}_{\text{mantisse}} e \underbrace{\pm 123}_{\text{exposant}}$$

pour $\pm 1,234\dots \times 10^{\pm 123}$. La *mantisse* est un nombre décimal (positif ou négatif) appartenant à $[1, 10[$ et l'*exposant* est un entier (lui aussi positif ou négatif). En Python la mantisse à une précision de 16 chiffres après la virgule.

Considérons la représentation de 2^{100} en entier et en notation scientifique :

[1] :

```
x = 2**100
y = 2.**100
print("x={} et y={}".format(x,y))
```

x=1267650600228229401496703205376 et y=1.2676506002282294e+30

La mantisse de 2^{100} est alors 1.2676506002282294 et son exposant est +30.

Pour éviter de manipuler des représentations différentes d'un même nombre, on convient de placer la virgule décimale après le premier chiffre non nul, d'où le nom notation scientifique. Les nombres réels sont représentés en Python avec le type *double précision* de la norme *IEEE754*. Les valeurs possibles pour l'exposant sont limitées ; approximativement, en deçà de 10^{-323} le nombre est arrondi à 0 ; au delà de 10^{308} , il y a dépassement de capacité : le nombre est considéré comme infini ($\text{inf} = \infty$).

```
[2]: x = 1e-324
      y = 1e-323
      z = 1e308
      w = 1e309
      print("x={} ; y={} ; z={} et w={}".format(x, y, z, w))
```

x=0.0 ; y=1e-323 ; z=1e+308 et w=inf

Plus précisément, le module `sys` permet d'accéder à la constante epsilon (ε) qui donne la différence entre 1.0 et le flottant suivant le plus proche :

```
[3]: from sys import float_info as fi
      x = fi.epsilon
      print("x={}".format(x))
```

x=2.220446049250313e-16

Notons que cette constante peut-être utilisée pour écrire une fonction qui *teste l'égalité* entre deux flottants dans la limite de la précision de l'interpréteur.

Travaux pratiques 5.

Les flottants sont représentés avec 16 significatifs ; taper un algorithme (code LaTeX) pour le vérifier.

2.2. Erreurs dues à la représentation

La tâche de représentation est plus complexe dans le cas des nombres réels. En effet, dans le système décimal, nous avons l'habitude de représenter un nombre x sous la forme

$$x = \pm m \times 10^l$$

où m est la mantisse, l est l'exposant et 10 est la base. De façon générale, selon une base b quelconque, on peut écrire :

$$x = \pm m \times b^l.$$

On appelle cette représentation la notation *flottante* ou notation en *point flottant* ou encore en *virgule flottante*. Puisqu'il n'est pas possible de mettre en mémoire une mantisse de longueur infinie, on recourt à la *troncature* ou à l'*arrondi* pour réduire la mantisse à n chiffres.

Un nombre réel x sera donc stocké en mémoire sous forme arrondie $\tilde{x}$ vérifiant :

$$\tilde{x} = x(1 + \varepsilon_x) \quad \text{avec} \quad |\varepsilon_x| \leq \varepsilon$$

Le nombre ε_x est alors appelé *erreur relative* et le nombre $x\varepsilon_x$ *erreur absolue*.

Les *problèmes d'arrondis* sont à prendre en compte dès que l'on travaille avec des nombres à virgule flottante. En général, pour éviter les *pertes de précision*, on essaiera autant que peut se faire de respecter les règles suivantes :

- éviter d'additionner ou de soustraire deux nombres d'ordres de grandeur très différente ;
- éviter de soustraire deux nombres presque égaux.

2.3. Principe : photo de classe

De plus, pour calculer une somme de termes ayant des ordres de grandeur très différents, on applique le principe dit *photo de classe* : les petits devant, les grands derrière. Par exemple, si on souhaite donner une approximation décimale de $\zeta(4) = \sum_{n=1}^{+\infty} \frac{1}{n^4}$ dont la valeur exacte est $\frac{\pi^4}{90}$, on constate qu'on obtient une meilleure précision en commençant par sommer les termes les plus petits :

```
[4]: from math import *

def zeta(n,p):
    return sum(1/i**p for i in range(1, n+1))

def zeta_inverse(n,p):
    return sum(1/i**p for i in range(n, 0, -1))

n, p = 2**18, 4
print('L'erreur du plus petit au plus grand :',
      "{: .16f}".format(abs(zeta_inverse(n, p) - pi**4/90)))

print('L'erreur du plus grand au plus petit :',
      "{: .16f}".format(abs(zeta(n, p) - pi**4/90)))
```

L'erreur du plus petit au plus grand : 0.00000000000000002

L'erreur du plus grand au plus petit : 0.000000000000002769